Journal of Physics:
Conference Series

PURPOSE-LED PUBLISHING™

**PAPER** • **OPEN ACCESS**

# Accelerating the Machine Learning Model Deployment using MLOps

To cite this article: Mandepudi Nobel Chowdary *et al* 2022 *J. Phys.: Conf. Ser.* **2327** 012027

View the article online for updates and enhancements.

You may also like

- Automatic machine learning Framework for Forest fire forecasting
Jintao Qu and Xiaohui Cui
- Prediction of vessel propulsion power using machine learning on AIS data, ship performance measurements and weather data
Q Liang, H A Tvete and H W Brinks
- Short-Term Wind Speed Prediction of Wind Farms Based on Particle Swarm Optimization Support Vector Machine
Xin Yu, Zhongxuan Zhang and Mingxuan Song

This content was downloaded from IP address 178.135.22.254 on 21/08/2025 at 12:26

# Accelerating the Machine Learning Model Deployment using MLOps

**Mandepudi Nobel Chowdary, Bussa Sankeerth, Chennupati Kumar Chowdary, ManuGupta***

Department of Electronics and Computer Engineering, Sreenidhi Institute of Science and Technology, Hyderabad, TS – 501 301, INDIA
*manugupta@sreenidhi.edu.in,manugupta5416@gmail.com
*orcid.org/0000-0003-4517-0129

**Abstract.** Machine Learning projects acquire a large amount of data to make predictions on the data. Deploying machine learning models is a difficult task as it is involved a lot factors such as continuous builds to improve efficiency and different libraries used for prediction. As the data grow for better predictions with proper accuracy some of the parameters are to be dynamically changed for the sake of performance tuning. To enhance the variation in the parameters it takes effort and time from the developers to make the changes in the code and deploy to the production environment. In proposed work, an end to end automation cycle to enhance the timely deployment of machine learning models with better performance is presented. This automation cycle converts the changed code with varied parameters into the container image and uploads to the container registry. Later, when the new image is uploaded it auto detects and updates the production environment with the new image data by launching a new container and shutting down the previous old one. The proposed automated model will display the updated features which are developed. The users can utilize the updated model simultaneously without any downtime to obtain the new features. The main contribution of proposed work is that it can build and deploy the machine learning models automatically, without any manual intervention. The models can also be retrained for better performance of the predictions.

**Keywords**: Machine Learning; Container; Deployment; Automation; Prediction.

## 1. Introduction

The DevOps in the IT sector helps to make the progress of deployment agile. It is very important to run the various production activities in every domain with agility. But it is very unfortunate as many numbers of the machine learning is not completed on the deadlines due to the lack of proper automation. The involvement of the DevOps activities in the line of machine learning practices will enable it to transform quickly into the production environment and can be used by the end users. Generally, machine learning is the combination of the data and algorithms to perform the operations on the data. DevOps can be combined with the machine learning resulting in the unique approach called as MLOps as shown in the figure 1. It is a complete cycle that involves the code development, model generation, orchestration and deployment. The overall life cycle is the complete combination of all the activities. MLOps helps to

Content from this work may be used under the terms of the Creative Commons Attribution 3.0 licence. Any further distribution of this work must maintain attribution to the author(s) and the title of the work, journal citation and DOI.
Published under licence by IOP Publishing Ltd

build the models in the most effective and efficient way with in the span of time enabling the businesses to be quick and agile [1]. Although MLOps is an intersection of the machine learning, data engineering and DevOps to enhance the better automated development equipped with the process of proper deployment. It enables the IT teams to enhance better view on the product by enabling the product deployment on the automated cycle with the concept of MLOps.

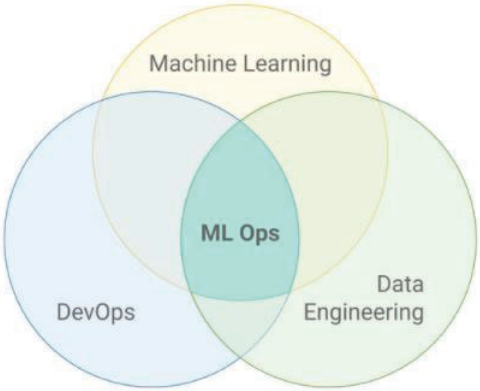


**Figure 1.** MLOps Intersection.

## 2. Literature survey

Machine Learning is divided into different categories. It starts with data collection, data processing, feature engineering, data labelling, model designing, model training and optimization by tuning the multiple parameters. Sometimes, it is quite often in the world of machine learning that data is trained multiple times recursively to enable the better accuracy of the model. This is a part of the development life cycle of machine learning products. It is very much necessary to update the features on the production environments as soon as possible by ensuring the proper working of multiple components. To enhance the faster deployment multiple DevOps tools are aligned in the line and connected with each other to enable the flow of the code from the developer to the production environment [2]. It is also necessary to deploy the code and design with the proper working and without any sort of errors.

The word MLOps defines the process of taking the experimented machine learning models to the production environment. It is the combination of work of the Data Scientists, DevOps engineers and the Machine Learning Engineer to ensure the transition from one area to the production environment. It improves the quality of the models in the production environment and enables automation to drive the business needs into better profits. It is a complete cycle that includes continuous development and continuous integration to support the multiple levels of automation.

DevOps enables the deployment on the cloud environments by transforming the code into the production builds of the product. Machine Learning is enabling the multiple software lifecycles to have the better agile stages of the life cycle and ensure proper planning with the deployment. DevOps itself contain a large range of life cycle practices that are to be optimized for better production builds. But although it can play a vital role in the development of the software product for the production builds [3]. This enables the unique concept of the MLOps to be introduced to enable the better flow of code from the developer to the deployment with the multiple levels of automation and connected by various pipelines. It is developed as the complete integration of the concept of machine learning involving a wide range of stages and combined with the stages of the deployment, thus enabling the combination as a single tool or technology for better optimization of the machine learning projects with frequent changes and tuning of the multiple parameters.

MLOps is a practice to be developed as a flow of a single entity that enables the building of better production systems. Many of the current DevOps practices are not really scalable for a large number of services as summarized in Table 1. Although the MLOps need the current practices of DevOps in the most automated way for better and faster implementation. The DevOps will enable the MLOps to be more useful and enhance better equipment of the resources. Modelling CI/CD pipelining through the usage of the various automation tools will ensure the better design of the MLOps pipeline [4]. Various kinds of environment designs are already present to overcome similar challenges in the deployment but a complete automated product or tool is necessary to speed up the ongoing machine learning projects with the overall integration of the multiple components.

**Table 1.** Feature Comparison

| Features | Existing Model | Proposed Model |
|---|---|---|
| Source Code Management [5] | Manual Code Upload | Auto upload |
| Repository Management [6] | Webhooks | Auto trigger |
| Automation [2] | Script based automation | Pipelined Automation |
| Scalability [7] | Manual scalability | Auto scalability |
| Container [8] | Manual deployment | Auto deployment |

## 3. Implementation

The implementation of the complete model with the integration of Machine Learning and DevOps require various tools of the two different domains and must be internally connected to ensure multiline automation from development to the deployment.

### *3.1. Machine Learning*

Machine Learning is the key technology to analyse the data and get the predictions out of it. Python acts as the most used programming language for the development of various machine learning algorithms. The deeper analysis of machine learning algorithms dives into the concept of neural networks which accounts for the domain of deep learning. There are many pre-developed libraries that help in building up the model. As a part of machine learning, Keras is used for training and building the model in this project. Keras is completely integrated with TensorFlow and enhances the developer to write the code for neural networks within a few lines. As a part of diabetes prediction, Keras is used as the neural network library to design the model that can intake the data and train it to make the predictions. The model is dependent on the multiple features which are provided as the input at the time of prediction that drives to the output..

TensorFlow is also used as a part of this project development. It is an open-source platform that enables the space and tools for the developers to design complex algorithms which can be helpful in solving the challenges faced by them. In this project, TensorFlow is used to perform the numerical calculation for building the model that is accurate and predicts the output for the given set of input features. It is also convenient and capable to integrate with python and develop the model accordingly with TensorFlow. TensorFlow is another additional library used for the development of the machine learning model in this project.

### *3.2. Docker*

The container is defined as the process on the operating system that provides a feature of another operating system running on the base operating system. The deployment of the applications will not be done directly on the base operating system in the real-time scenario due to the various reasons that might lead to different problems and failures. Hence, an extra operating system environment is created with the help of the tools like Docker and run the applications on top of it. Even during the time of failures that particular container can be shut down and start the new one which holds the same application. In this process of implementation, the base operating system remains unaffected. Apart from that container is light weighted operating system that provides abstraction across various other processes running along with it [7]. Docker acts as a tool that helps to install the container software on the system and helps to implement it in an easier manner with its architecture as shown in figure 2.

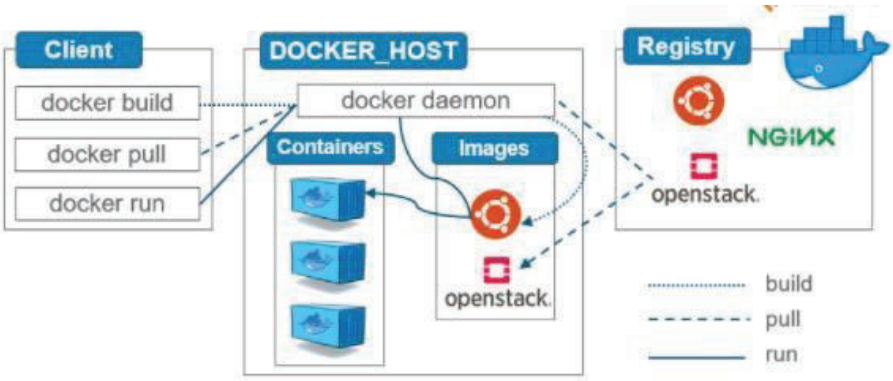


**Figure 2.** Docker Architecture.

Figure 2 describes the working of Docker in the client-server architecture. It enhances that the Docker host is the server and the client is the user, who can communicate to the server for various implementations. The server connects to the registry and gets the details that are necessary to fulfil the client request. In this proposed work, Docker is used in writing the Docker File that acts as an ISO image of an operating system.

### *3.3. Flask Framework*

Flask is the framework useful in developing web applications and running them as the webserver. It is completely designed with the python programming language. Flask is used in this project to develop the GUI console for the user to enter the feature details of the machine learning that can predict the chances of diabetes for the person with those habits. The Flask application is integrated with the machine learning model, where it intakes the data and predict the output by sending the data to the model and finally displays it on the screen, where the user can receive it. Since the application is very much compatible and designed with the features of security, Flask is used for web application deployment purposes. It also enables the developer to easily integrate with the other tools, as it is developed with the well-used programming language.

### *3.4. Elastic Container Registry*

Amazon Web Services provides an exclusive service similar to that of the Docker Hub that stores the ISO image files of the containers. This registry can be used with the various other integrated services and makes the process of deployment easier and faster. As a part of this project, ECR is used for storing the container image file which is developed and pushed by the developer that contains the various details

to launch the container, run the web application and work accordingly with the mentioned ports and the protocols. It is used apart from the Docker Hub is to enhance the deployment of the complete project on the cloud environment. Hence, the inbuilt services of the cloud are used for better accuracy and availability. The image is uploaded to ECR accordingly as shown in figure 3.

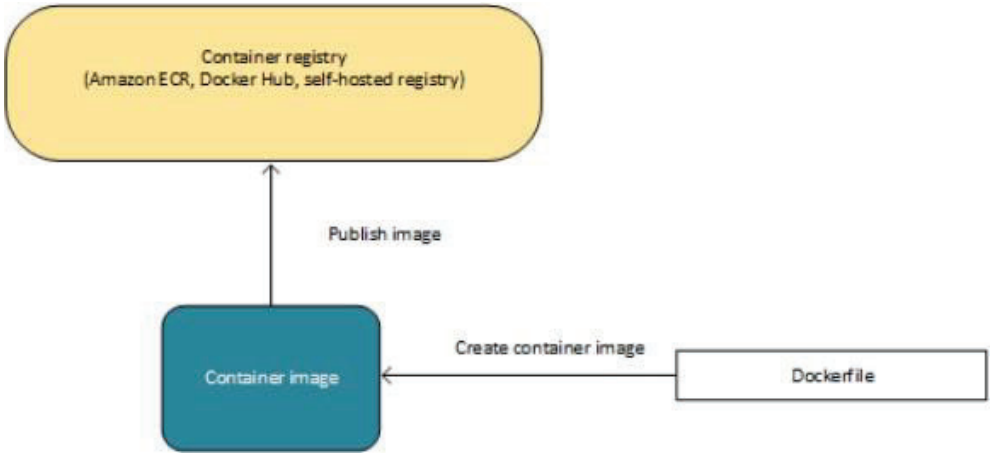


**Figure 3.** Image publishing using Docker to ECR.

### *3.5. Elastic Container Service*

Elastic Container Service is provided by the AWS to deploy the containers automatically on the base operating system without managing it by the user. The cloud looks after the base operating system and its resources of it. In that case, we can easily launch the containers based on our requirements and perform the deployment of the application on the containers. As a part of this project, the ECS is useful to launch the containers that hold the web application through which the user can communicate to make the predictions with the model deployed. The ECS is completely managed by the AWS, hence there will not be any sort of management of the operating systems by us externally. The ECS distributes the launching of the containers accordingly as shown in figure 4.

**Figure 4.** Task Scheduling using ECS.

### *3.6. Terraform*

Terraform is the tool for multi-cloud deployment. It helps to deploy the resources on the cloud environment with a piece of code. It plans the infrastructure prior to the deployment and it also has the feature to destroy it completely, even the multiple services are dependent. In this work, Terraform is used for the deployment of the infrastructure on the cloud. It looks after the resources related to the ECS, ECR and deployment of the containers. It manages the deployment of the infrastructure within the span of minutes and helps in the complete automation of the deployment process on the cloud. It can be used for infrastructure deployment in various cloud environments [9].

## 4. Architecture

The architecture describes the flow of the process and the various integrations of both the domains through multiple pipelines. It starts from the code development and automates all the processes till deployment on the server.

### *4.1. Model Building*

The initial step in the process of machine learning deployment is model building. The model building is performed based on the set of the lines in the Docker File. The model building requires the dataset and the pre-built libraries which are necessary for building the model. The process of model building starts with the training of the data to the algorithm and develops the model. The final model is produced as the h5 file. In this project, the model building is performed on the container environment, where it contains all the required dependencies that are mentioned through the Docker File. When the container is deployed with the Docker File, it is launched and runs the process of the model building. The flow of the code in the MLOps pipeline is defined as shown in figure 5.

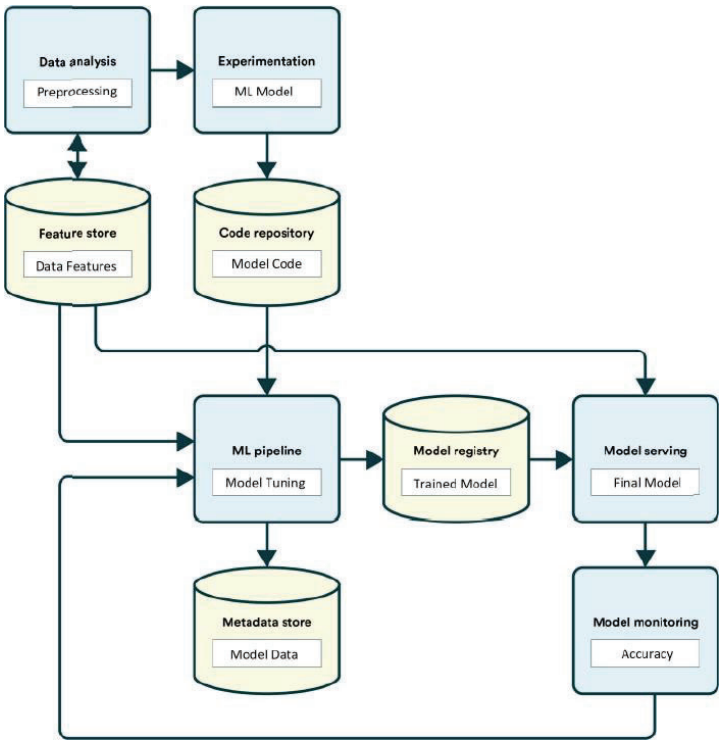


**Figure 5.** Block diagram of proposed model.

The dataset is read and pre-processing is done on the data. From the Keras library Sequential model, dense layer, Adam optimizer are imported. Here we used the multi-layered neural network which consists of 3 hidden layers, the input layer and hidden layers are activated by “relu” activation function while output layer is activated by “sigmoid” activation function. The “binary_crossentroy” function is used as the loss function and “adam” as the optimizer, this will give the accuracy as the metrics. The generated model has achieved accuracy of 73%, also model is repeatedly trained with the data for 100 times which helped in increasing the accuracy. At last, the model is saved with the name “cardio.h5” as shown in figure 6.

```
In [17]: model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

In [18]: model.fit(X,y, epochs=100)
         Epoch 91/100
         2188/2188 [==============================] - 2s 786us/step - loss: 0.5486 - accuracy: 0.7308
         Epoch 92/100
         2188/2188 [==============================] - 2s 863us/step - loss: 0.5528 - accuracy: 0.7273
         Epoch 93/100
         2188/2188 [==============================] - 2s 853us/step - loss: 0.5503 - accuracy: 0.7252
         Epoch 94/100
         2188/2188 [==============================] - 2s 776us/step - loss: 0.5477 - accuracy: 0.7298
         Epoch 95/100
         2188/2188 [==============================] - 2s 798us/step - loss: 0.5486 - accuracy: 0.7286
         Epoch 96/100
         2188/2188 [==============================] - 2s 783us/step - loss: 0.5495 - accuracy: 0.7289
         Epoch 97/100
         2188/2188 [==============================] - 2s 751us/step - loss: 0.5493 - accuracy: 0.7290
         Epoch 98/100
         2188/2188 [==============================] - 2s 810us/step - loss: 0.5500 - accuracy: 0.7277
         Epoch 99/100
         2188/2188 [==============================] - 2s 865us/step - loss: 0.5495 - accuracy: 0.7296
         Epoch 100/100
         2188/2188 [==============================] - 2s 869us/step - loss: 0.5471 - accuracy: 0.7318

In [19]: model.save("cardio.h5")
```

**Figure 6.** Model Training.

### *4.2. Creating a Docker image*

The Docker Image is similar to the ISO image, which contains the operating system details and the various process to be run when the operating system is launched in the form of a container. The Docker Image can be used to contain the details of all the processes to be run in the container once it is deployed or in the state of running. Various libraries that are necessary for the container which runs the web application in this project are included in the Docker Image. The Docker image file of this project and the flask application code is shown in figure 7 and figure 8.

```
FROM docker.io/library/centos

RUN yum install python3 -y

RUN yum install python3-devel -y

RUN yum install gcc-c++ -y

RUN pip3 install --upgrade pip

RUN pip3 install Flask

RUN pip3 install pandas

RUN pip3 install h5py

RUN pip3 install keras

RUN pip3 install tensorflow

EXPOSE 80
```

**Figure 7.** Docker Image.

```
from keras.models  import load_model
from flask import Flask, render_template, request

app = Flask("cardio_app")
model  =  load_model("cardio.h5")

@app.route("/home")
def home():
    return render_template("form.html")

@app.route("/output" , methods=[ "GET" ] )
def dia():
        x1 = request.args.get("age")
        x2 = request.args.get("gender")
        x3 = request.args.get("height")
        x4 = request.args.get("weight")
        x5 = request.args.get("systolic")
        x6 = request.args.get("diastolic")
        x7 = request.args.get("cholesterol")
        x8 = request.args.get("glucose")
        x9 = request.args.get("smoke")
        x10 = request.args.get("alcohol")
        x11 = request.args.get("activity")

        output = model.predict([[ int(x1) , int(x2) , int

        if (round(output[0][0])) == 1:
            return render_template("positive.html")

        else:
            return render_template("negative.html")

app.run(host="0.0.0.0" ,  port=80)
```

**Figure 8.** Flask Application.

### *4.3. Deploying Infrastructure using Terraform*

The Docker File defines the set of the programs and the processes to be run on the container. Once the container is launched the set of commands run in the container to run the processes [8]. It is uploaded to the ECR by the developer. Once the Docker File is uploaded to the ECR registry, it triggers to run the file. It launches the containers using the ECS and runs the multiple containers as a cluster [10]. The containers hold the web application through which the users can interact. The multiple clusters of the containers are load balanced using the Elastic Load Balancer service of the AWS. The ELB runs the web application on port 80 and uses the HTTP protocol as mentioned in the Docker File. Finally, the terraform code returns the DNS name of the load balancer. Using the URL the users can reach the application.

The entire deployment process is run using the Terraform. It enhances to run the complete process and provide the output whether the implementation is successful or not. In this way, the power of infrastructure as a code is utilized with the terraform as a tool.

## 5. Results

The complete MLOps flow will create the production environment to deploy the machine learning model. As a part of the model, the page is displayed to take the input details from the user. The entire application is made with the Flask and make the predictions based on the data trained to the model. The homepage displayed for front-end form is shown in the figure 9. The output obtained can be positive or negative as shown in the figure 10 and figure 11 respectively. Also, the updates in the code made by the developer and pushed to the source code repository will build up the production environment within the span of 3 minutes as shown in figure 12. The proposed work's key contribution is that it can automatically construct and deploy machine learning models without any operator intervention. The models can also be retrained to improve predictions accuracy.

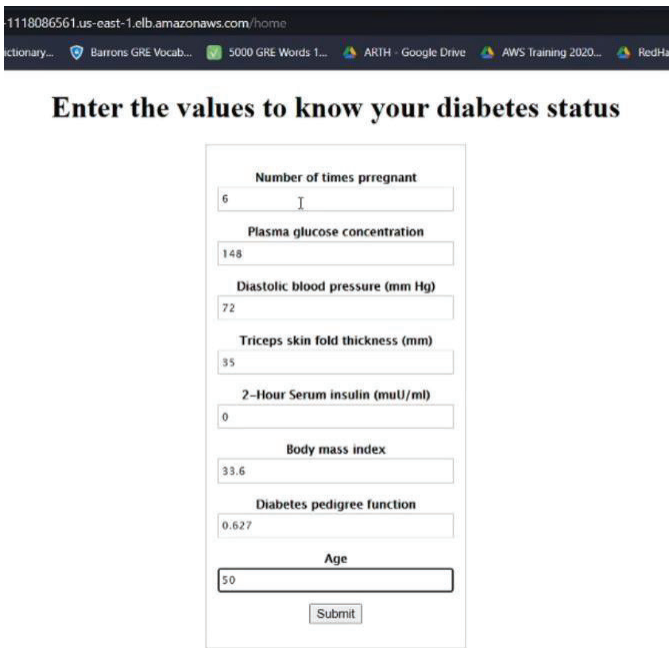


**Figure 9.** Front-end Form.

**Figure 10.** Positive Output.

**Figure 11.** Negative Output.

```
aws_lb.cluster_lb: Still creating... [2m30s elapsed]
aws_lb.cluster_lb: Still creating... [2m40s elapsed]
aws_lb.cluster_lb: Still creating... [2m50s elapsed]
aws_lb.cluster_lb: Still creating... [3m0s elapsed]
aws_lb.cluster_lb: Creation complete after 3m1s [id=arn:aws
/cluster-lb/77657c66c0abc6ea]
```

**Figure 12.** Timespan for Deployment

## 6. Conclusion and future scope

MLOps or DevOps for machine learning, empowers data science and IT groups to work together and increment the speed of model turn of events and arrangement through monitoring, validation and governance of machine learning models. MLOps is displayed on the current control of DevOps, the cutting-edge practice of effectively writing, deploying and running venture applications. DevOps got its beginning 10 years prior as a way fighting clans of software developers (the Dev) and IT tasks groups (the Ops) could team up. MLOps adds to the group the data scientists, which minister datasets and construct AI models that dissect them. It likewise incorporates ML engineers, who run those datasets through the models in focused, automated ways. When a ML model has been assembled, it should be coordinated with real-world data and the business application or front-end services. The entire application or parts thereof should be deployed without disrupting the assistance. Sending can be very difficult if the ML components aren't treated as a vital piece of the application or production pipeline [11]. The proposed model supports the model creation with automated machine learning and access incredible feature engineering, algorithm selection and hyper parameter-sweeping capabilities. This model increases group proficiency with shared datasets, notebooks, models and customizable dashboards that track all parts of the machine learning measure.

In the future, the model can be improved by performing the tuning of the parameters and analysing the accuracy of the result with the multiple parameters. In this way, it can also be integrated with tool such as Kubernetes to deploy the containers efficiently and perform the scaling of them and other features of monitoring their health and maintaining the high availability [12]. This process of MLOps will enable the data scientists to work more on the better model building and the deployment is completely automated which enhances the retraining of the model for better accuracy.

## References

[1] Singla K, Bose J and Naik C 2018 Analysis of software engineering for agile machine learning projects *15th IEEE India Council International Conference (INDICON)* (Coimbatore, India) pp 1-5.

[2] Guşeilă L G, Bratu D V and Moraru S A 2019 DevOps transformation for multi-cloud IoT applications *IEEE International Conference on Sensing and Instrumentation in IoT Era (ISSI)* (Lisbon, Portugal) pp 1-6.

[3] Shafiq S, Mashkoor A, Mayr-Dorn C and Egyed A 2021 A Literature Review of Machine Learning and Software Development Life cycle Stages *IEEE Access* **9**, pp 140896-140920.

[4] Mäkinen S, Skogström H, Laaksonen E and Mikkonen T 2021 Who Needs MLOps: What Data Scientists Seek to Accomplish and How Can MLOps Help? *IEEE/ACM 1st Workshop on AI Engineering-Software Engineering for AI (WAIN)* (Madrid, Spain) pp 109-112.

[5] Gupta M, Chowdary M N, Bussa S and Chowdary C K 2021 Deploying Hadoop Architecture Using Ansible and Terraform *5th IEEE International Conference on Information Systems and Computer Networks (ISCON)* (Mathura, India) pp 1-6.

[6] Gupta M, Sanjana K, Akhilesh K and Chowdary M N 2021 Deployment of Multi-Tier Application on Cloud and Continuous Monitoring using Kubernetes *5th IEEE International Conference on Electrical, Electronics, Communication, Computer Technologies and Optimization Techniques (ICEECCOT)* (Mysuru, India) pp 602-607.

[7] Tamburri D A 2020 Sustainable MLOps: Trends and challenges *22nd IEEE International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC)* (Timisoara, Romania) pp 17-23.

[8] Bates W and Oviedo E I 2021 Software Reliability in a DevOps Continuous Integration Environment *Annual Reliability and Maintainability Symposium (RAMS)* (Orlando, FL, USA) pp 1-4.

[9] Marijan D, Liaaen M and Sen S 2018 DevOps improvements for reduced cycle times with integrated test optimizations for continuous integration *42nd IEEE Annual computer software and applications conference (COMPSAC)* vol 1 (Tokyo, Japan) pp 22-27.

[10] Liao Q 2020 Modelling CI/CD Pipeline Through Agent-Based Simulation *IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)* (Coimbra, Portugal) pp 155-156.

[11] Mysari S and Bejgam V 2020 Continuous Integration and Continuous Deployment Pipeline Automation Using Jenkins Ansible *IEEE International Conference on Emerging Trends in Information Technology and Engineering (ic-ETITE)* (Vellore, India) pp 1-4.

[12] Singh C, Gaba N S, Kaur M and Kaur B 2019 Comparison of different CI/CD tools integrated with cloud platform *9th IEEE International Conference on Cloud Computing, Data Science & Engineering (Confluence)* (Noida, India) pp 7-12.